

Lab 5 — Securing Apache Web Server

Part 0 — Check prerequisites

First check OpenSSL and Apache:

```
openssl version  
apache2 -v
```

You should get something similar to:

```
OpenSSL 3.x.x  
Server version: Apache/2.4.x
```

If Apache is not installed:

```
sudo apt update  
sudo apt install apache2 openssl -y
```

Then:

```
sudo systemctl start apache2  
sudo systemctl enable apache2
```

Task 1 — Become a Certificate Authority

The lab requires you to create a Root CA and use it to issue certificates. The CA has a private key (`ca.key`) and certificate (`ca.crt`).

Step 1: Create Lab5 directory

```
mkdir -p ~/Lab5/CA  
cd ~/Lab5/CA
```

Copy the OpenSSL configuration:

```
cp /usr/lib/ssl/openssl.cnf .
```

Check:

```
ls
```

You should see:

```
openssl.cnf
```

Step 2: Create required directories

The lab's OpenSSL configuration uses several CA directories. Create them:

```
mkdir -p certs newcerts private  
touch index.txt  
echo 1000 > serial
```

Check:

```
ls -la
```

You should have:

```
certs  
newcerts  
private  
index.txt  
openssl.cnf  
serial
```

Step 3: Generate Root CA

Run:

```
openssl req -new -x509 -keyout ca.key -out ca.crt -config openssl.cnf
```

The lab specifically instructs this command.

You will be asked for a password.

Remember this password.

For example:

Enter PEM pass phrase:

Verifying - Enter PEM pass phrase:

Then certificate information.

You can use:

Country Name: BD

State or Province Name: Sylhet

Locality Name: Sylhet

Organization Name: SUST

Organizational Unit Name: CSE Lab 5

Common Name: Hashem Arif

Email Address: your email

Then check:

```
ls -l ca.*
```

You should have:

```
ca.crt  
ca.key
```

Task 2 — Create certificate for **example.com**

The lab creates the server certificate in three steps: generate a key, generate a CSR, and have the CA sign it.

Step 1 — Generate server private key

The lab gives:

```
openssl genrsa -des3 -out server.key 1024
```

However, because your current OpenSSL is **3.0.13**, a 1024-bit RSA key can cause problems. For your actual Ubuntu system, use:

```
openssl genrsa -aes256 -out server.key 2048
```

Enter a password.

Check:

```
ls -l server.key
```

Step 2 — Generate CSR

Run:

```
openssl req -new -key server.key -out server.csr -config openssl.cnf
```

When asked for:

Common Name

IMPORTANT:

Enter:

example.com

The lab specifically requires `example.com` as the common name.

Check:

ls -l server.csr

Step 3 — Sign the certificate using your CA

Run:

```
openssl ca -in server.csr -out server.crt -cert ca.crt -keyfile ca.key -config openssl.cnf
```

Enter your CA password.

If successful, you should get:

Certificate is to be certified until ...

Sign the certificate? [y/n]

Enter:

y

Then:

1 out of 1 certificate requests certified, commit? [y/n]

Enter:

y

Now:

ls -l server.*

You should have:

server.key

server.csr

server.crt

Task 2 — Test HTTPS using OpenSSL

The lab says to combine the server private key and certificate into `server.pem`.

Run:

```
cp server.key server.pem  
cat server.crt >> server.pem
```

Then:

```
openssl s_server -cert server.pem -www
```

You should see something like:

```
Using default temp DH parameters  
ACCEPT
```

The OpenSSL server listens on port **4433** by default.

Step 3 — Add `example.com` to `/etc/hosts`

Open another terminal.

Check:

```
grep -n "example.com" /etc/hosts
```

If it isn't there:

```
echo "127.0.0.1 example.com" | sudo tee -a /etc/hosts
```

Then:

```
grep -n "example.com" /etc/hosts
```

You should see:

```
127.0.0.1 example.com
```

Step 4 — Test in browser

Open:

<https://example.com:4433>

Initially, Firefox/Chrome may show a certificate warning because your CA is not trusted yet. This is **expected** according to the lab.

Step 5 — Install your CA certificate

For Ubuntu system trust, copy your CA certificate:

```
sudo cp ca.crt /usr/local/share/ca-certificates/lab5-ca.crt
```

Then:

```
sudo update-ca-certificates
```

You should see something similar to:

```
1 added, 0 removed
```

Restart Firefox/Chrome if necessary.

Now visit:

<https://example.com:4433>

The certificate should be trusted.

Checkpoint 1

Show your teacher:

<https://example.com:4433>

Explain:

1. `ca.crt` is your Root CA certificate.
2. `server.crt` was signed by your CA.
3. Browser verifies the server certificate.
4. TLS establishes an encrypted connection.
5. The browser and server communicate over HTTPS.

The lab identifies this as **Checkpoint 1 — 5 marks**.

Task 2 — **webserverlab.com**

Now repeat the certificate process for:

webserverlab.com

Add it to **/etc/hosts**:

```
echo "127.0.0.1 webserverlab.com" | sudo tee -a /etc/hosts
```

Create another key:

```
openssl genrsa -aes256 -out webserverlab.key 2048
```

Create CSR:

```
openssl req -new -key webserverlab.key -out webserverlab.csr -config openssl.cnf
```

For:

Common Name

enter:

webserverlab.com

Sign it:

```
openssl ca -in webserverlab.csr -out webserverlab.crt -cert ca.crt -keyfile ca.key -config openssl.cnf
```

Create PEM:

```
cp webserverlab.key webserverlab.pem  
cat webserverlab.crt >> webserverlab.pem
```

Start the OpenSSL server using another port:

```
openssl s_server -cert webserverlab.pem -www -accept 4443
```

Then visit:

<https://webserverlab.com:4443>

Checkpoint 2

Show the HTTPS page for:

<https://webserverlab.com:4443>

The lab explicitly requires this as Checkpoint 2.

Task 3 — Deploy HTTPS in Apache

First stop the OpenSSL server.

Go to the terminal running:

```
openssl s_server
```

Press:

Ctrl + C

The lab specifically says to stop the OpenSSL web server before configuring Apache.

Step 1 — Enable SSL module

Run:

```
sudo a2enmod ssl
```

Then:

```
sudo systemctl restart apache2
```

The lab explicitly requires `a2enmod ssl`.

Step 2 — Copy certificates

Create a directory:

```
sudo mkdir -p /etc/apache2/ssl
```

Copy certificate:

```
sudo cp ~/Lab5/CA/server.crt /etc/apache2/ssl/example.com.crt
```

Copy private key:

```
sudo cp ~/Lab5/CA/server.key /etc/apache2/ssl/example.com.key
```

Check:

```
ls -l /etc/apache2/ssl/
```

You should see:

```
example.com.crt  
example.com.key
```

Step 3 — Configure **example.com**

Open:

```
sudo nano /etc/apache2/sites-available/example.com.conf
```

Add:

```
<IfModule mod_ssl.c>  
<VirtualHost *:443>  
  
    ServerAdmin admin@example.com  
    ServerName example.com  
    ServerAlias www.example.com  
  
    DocumentRoot /var/www/example.com/html  
  
    ErrorLog ${APACHE_LOG_DIR}/example-error.log  
    CustomLog ${APACHE_LOG_DIR}/example-access.log combined  
  
    SSLEngine on  
    SSLCertificateFile /etc/apache2/ssl/example.com.crt  
    SSLCertificateKeyFile /etc/apache2/ssl/example.com.key  
  
</VirtualHost>  
</IfModule>
```

This follows the configuration structure given by the lab, with the placeholder certificate paths replaced by your actual certificate/key paths.

Save:

```
Ctrl + O  
Enter  
Ctrl + X
```

Step 4 — Enable the HTTPS site

Run:

```
sudo a2ensite example.com.conf
```

Then:

```
sudo apache2ctl configtest
```

You want:

Syntax OK

The lab explicitly uses `apache2ctl configtest` as the configuration check.

Then:

```
sudo systemctl restart apache2
```

Check:

```
sudo systemctl status apache2
```

You want:

active (running)

Step 5 — Test Apache HTTPS

Open:

```
https://example.com
```

If your CA is trusted, the browser should open the site without the previous certificate warning.

Checkpoint 3 — 5 marks

Show:

```
https://example.com
```

to your teacher.

The lab specifically identifies this as Checkpoint 3.

Task 3 — **webserverlab.com**

Now configure the second virtual host.

Copy its certificate:

```
sudo cp ~/Lab5/CA/webserverlab.crt /etc/apache2/ssl/
```

Copy its key:

```
sudo cp ~/Lab5/CA/webserverlab.key /etc/apache2/ssl/
```

Open:

```
sudo nano /etc/apache2/sites-available/webserverlab.com.conf
```

Use:

```
<IfModule mod_ssl.c>
```

```
<VirtualHost *:443>
```

```
    ServerAdmin admin@webserverlab.com
```

```
    ServerName webserverlab.com
```

```
    ServerAlias www.webserverlab.com
```

```
    DocumentRoot /var/www/webserverlab.com/html
```

```
    ErrorLog ${APACHE_LOG_DIR}/webserverlab-error.log
```

```
    CustomLog ${APACHE_LOG_DIR}/webserverlab-access.log combined
```

```
    SSLEngine on
```

```
    SSLCertificateFile /etc/apache2/ssl/webserverlab.crt
```

```
    SSLCertificateKeyFile /etc/apache2/ssl/webserverlab.key
```

```
</VirtualHost>
```

```
</IfModule>
```

Enable it:

```
sudo a2ensite webserverlab.com.conf
```

Check configuration:

```
sudo apache2ctl configtest
```

Expected:

Syntax OK

Restart:

```
sudo systemctl restart apache2
```

Step 6 — Test **webserverlab.com**

Make sure `/etc/hosts` contains:

```
127.0.0.1 example.com
127.0.0.1 webserverlab.com
```

Then open:

```
https://webserverlab.com
```

Checkpoint 4 — 5 marks

Show:

```
https://webserverlab.com
```

to your teacher.

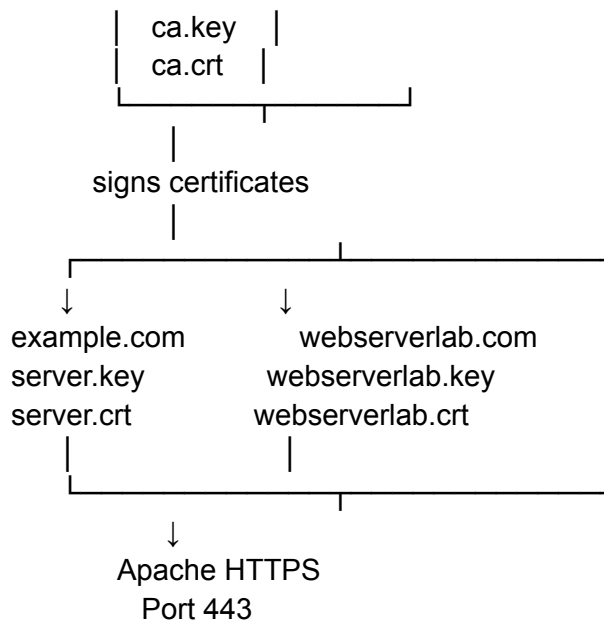
Final checkpoint summary

Checkpoint	What you show	Marks
1	<code>https://example.com:4433</code> using OpenSSL server	5
2	<code>https://webserverlab.com:4443</code> using OpenSSL server	5
3	<code>https://example.com</code> using Apache HTTPS	5
4	<code>https://webserverlab.com</code> using Apache HTTPS	5
Total		20

The important architecture is:

Your Root CA





One important point for your machine: because you're using modern OpenSSL 3.x, I would **not blindly use the lab's 1024-bit RSA command**. The lab itself gives 1024, but modern OpenSSL/Apache setups can reject weak keys. The 2048-bit commands above avoid that problem while preserving the lab's objective.

Lab 6: Securing Apache Web Server – 2

This lab extends **Lab 5**. You already have:

`https://example.com`
`https://webserverlab.com`

Now you will add:

1. **HTTP → HTTPS redirection**
2. **File-based authentication using `.htpasswd`**
3. **MySQL database-based authentication**

There are **3 checkpoints**: **5 + 7 + 8 = 20 marks**.

Before starting

Make sure Apache is running:

```
sudo systemctl status apache2
```

If it is not running:

```
sudo systemctl start apache2
```

Also verify your sites:

```
ls /etc/apache2/sites-enabled/
```

You should have something related to:

```
example.com.conf  
webserverlab.com.conf
```

Your Lab 5 HTTPS configuration should already be working.

TASK 1 — Redirect HTTP to HTTPS

5 Marks

The purpose is:

http://example.com



Apache mod_rewrite



https://example.com

The lab specifically asks you to use `mod_rewrite`.

Step 1 — Enable mod_rewrite

Run:

```
sudo a2enmod rewrite
```

You should see something similar to:

Enabling module rewrite.

To activate the new configuration, you need to run:

```
systemctl restart apache2
```

Step 2 — Find the HTTP configuration

Run:

```
ls -l /etc/apache2/sites-enabled/
```

Then find `example.com`:

```
grep -Rni "ServerName example.com" /etc/apache2/sites-enabled/
```

You may get:

```
/etc/apache2/sites-enabled/example.com.conf:...
```

Open the configuration:

```
sudo nano /etc/apache2/sites-available/example.com.conf
```

Important

You need to put the rewrite rules **inside the `<VirtualHost *:80>` block**, not inside the HTTPS `<VirtualHost *:443>` block.

It should look approximately like this:

```
<VirtualHost *:80>

    ServerAdmin admin@example.com
    ServerName example.com
    ServerAlias www.example.com

    DocumentRoot /var/www/example.com/html

    ErrorLog ${APACHE_LOG_DIR}/example-error.log
    CustomLog ${APACHE_LOG_DIR}/example-access.log combined

    RewriteEngine On
    RewriteCond %{HTTPS} !=on
    RewriteRule ^/?(.*) https://%{SERVER_NAME}/$1 [R,L]

</VirtualHost>
```

The three important lines from your lab are:

```
RewriteEngine On
RewriteCond %{HTTPS} !=on
RewriteRule ^/?(.*) https://%{SERVER_NAME}/$1 [R,L]
```

Step 3 — Test Apache configuration

Run:

```
sudo apache2ctl configtest
```

Expected:

Syntax OK

If you get anything other than **Syntax OK**, **don't restart Apache yet**. Send me the error.

Step 4 — Restart Apache

```
sudo systemctl restart apache2
```

Check:

```
sudo systemctl status apache2
```

You want:

active (running)

Step 5 — Test

Open:

`http://example.com`

It should automatically become:

`https://example.com`

You can also test from terminal:

`curl -I http://example.com`

You should see a redirect, typically:

HTTP/1.1 301 Moved Permanently
Location: `https://example.com/...`

CHECKPOINT 1 — 5 Marks

Show your teacher:

`http://example.com`

and demonstrate that Apache automatically redirects it to:

`https://example.com`

TASK 2 — Basic Authentication with `.htpasswd`

7 Marks

Now only authenticated users will be able to access:

`https://example.com`

Apache's `htpasswd` utility creates the username/password file, and Apache's `file` authentication provider uses that file.

Step 1 — Create first user

Run:

```
sudo htpasswd -c /etc/apache2/.htpasswd sammy
```

It will ask:

New password:

Re-type new password:

Enter a password you will remember.

For example, don't literally use the lab's sample password; create your own.

Step 2 — Add second user

Do NOT use `-c` again.

For example:

```
sudo htpasswd /etc/apache2/.htpasswd alice
```

Enter Alice's password.

Your file now contains two users.

Step 3 — View `.htpasswd`

Run:

```
sudo cat /etc/apache2/.htpasswd
```

You'll see something like:

```
sammy:$apr1$.....$.....  
alice:$apr1$.....$.....
```

The exact hashes will be different.

Don't worry about the hash being different from your friend's—the password hash is normally salted.

Also, keeping the password file outside the web document tree is important; Apache's documentation explicitly recommends that it not be directly fetchable through the website.

Step 4 — Configure Apache authentication

Open your HTTPS configuration:

```
sudo nano /etc/apache2/sites-available/example.com.conf
```

Inside your **HTTPS** `<VirtualHost *:443>`, add:

```
<Directory "/var/www/example.com/html">
    AuthType Basic
    AuthName "Restricted Content"
    AuthUserFile /etc/apache2/.htpasswd
    Require valid-user
</Directory>
```

So your HTTPS configuration will look approximately like:

```
<IfModule mod_ssl.c>
```

```
<VirtualHost *:443>
```

```
    ServerAdmin admin@example.com
    ServerName example.com
    ServerAlias www.example.com
```

```
    DocumentRoot /var/www/example.com/html
```

```
    ErrorLog ${APACHE_LOG_DIR}/example-error.log
    CustomLog ${APACHE_LOG_DIR}/example-access.log combined
```

```
    SSLEngine on
    SSLCertificateFile /etc/apache2/ssl/example.com.crt
    SSLCertificateKeyFile /etc/apache2/ssl/example.com.key
```

```
    <Directory "/var/www/example.com/html">
        AuthType Basic
        AuthName "Restricted Content"
        AuthUserFile /etc/apache2/.htpasswd
        Require valid-user
    </Directory>
```

```
</VirtualHost>
```

</IfModule>

What these lines do

AuthType Basic

Uses HTTP Basic Authentication.

AuthName "Restricted Content"

Sets the authentication realm/message.

AuthUserFile /etc/apache2/.htpasswd

Tells Apache where the users/password hashes are stored.

Require valid-user

Allows any user whose credentials successfully authenticate.

This is consistent with Apache's documented file-based authentication configuration.

Step 5 — Test configuration

sudo apache2ctl configtest

Expected:

Syntax OK

Then:

sudo systemctl restart apache2

Step 6 — Test authentication

Open:

https://example.com

You should now get a popup similar to:

Authentication Required

Username:

Password:

Enter:

Username: sammy

Password: [the password you created]

You should now get access to the website.

Try also:

alice

with Alice's password.

✓ CHECKPOINT 2 — 7 Marks

Show your teacher:

<https://example.com>

Then demonstrate:

Username + Password



Authentication successful



Website accessible

TASK 3 — MySQL Database Authentication

8 Marks

Now we're going to remove the `.htpasswd` authentication and make Apache authenticate users from:

MySQL



apache database



users table

Apache's `mod_authn_dbd` is specifically designed to authenticate users from an SQL database.

Step 1 — Install MySQL

Run:

```
sudo apt update
```

Then:

```
sudo apt install mysql-server -y
```

The lab says to install the MySQL server.

Then install the Apache MySQL database driver:

```
sudo apt install libaprutil1-dbd-mysql -y
```

Important: your pasted lab has:

```
sudo apt-get installlibaprutil1-dbd-mysql
```

There is a missing space in that command. Use:

```
sudo apt install libaprutil1-dbd-mysql -y
```

Step 2 — Secure MySQL

Run:

```
sudo mysql_secure_installation
```

Follow your lab's instructions.

However, on modern Ubuntu, the authentication method for the MySQL `root` account can differ from older lab environments. So if the lab's exact `root` password behavior doesn't appear, **don't randomly change MySQL authentication settings**—send me the prompt you get.

Step 3 — Check MySQL

```
systemctl status mysql.service
```

You want:

active (running)

If it is running, continue.

Step 4 — Login to MySQL

According to the lab:

```
mysql -u root -p
```

Enter your MySQL root password.

You should get:

```
mysql>
```

Step 5 — Create database

Inside MySQL:

```
CREATE DATABASE apache;
```

Then:

```
USE apache;
```

Step 6 — Create users table

Run:

```
CREATE TABLE users (  
    username VARCHAR(30) PRIMARY KEY,  
    password VARCHAR(512) NOT NULL  
);
```

Check:

```
SHOW TABLES;
```

You should see:

```
users
```

Step 7 — Generate password hash

Open **another terminal**.

Run:

```
htpasswd -nbs sammy sammys_password
```

You will get something like:

```
sammy:{SHA}xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx=
```

The exact value will be different.

The lab tells you to copy everything after the `:` including:

```
{SHA}
```

Apache documents this `{SHA}` format as SHA-1 + Base64; it is suitable for this lab's DBD authentication example, although Apache currently considers SHA-1 password storage insecure for real deployments.

Step 8 — Insert Sammy into MySQL

Go back to MySQL:

```
mysql -u root -p
```

Then:

```
USE apache;
```

Suppose your generated hash was:

```
{SHA}xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx=
```

Then:

```
INSERT INTO users VALUES  
( 'sammy', '{SHA}xxxxxxxxxxxxxxxxxxxxxxxxxx=' );
```

Do not copy my fake hash.

Use the hash that your own command generated.

Step 9 — Create Alice

In another terminal:

```
htpasswd -nbs alice alice_password
```

You'll get:

```
alice:{SHA}.....
```

Copy everything after `:`.

Then in MySQL:

```
INSERT INTO users VALUES  
( 'alice', '{SHA}.....' );
```

Check your database:

```
SELECT * FROM users;
```

You should see:

```
+-----+-----+  
| username | password |  
+-----+-----+  
| sammy   | {SHA}..... |  
| alice   | {SHA}..... |  
+-----+-----+
```

Step 10 — Enable Apache DBD modules

Exit MySQL:

```
exit
```

Then run:

```
sudo a2enmod dbd
sudo a2enmod authn_dbd
sudo a2enmod socache_shmcb
sudo a2enmod authn_socache
```

The purpose is to make Apache's database-backed authentication available.

Step 11 — Configure Apache to connect to MySQL

This is the most important part of Task 3.

Your pasted lab says:

"replace it with the following contents"

but **the actual configuration block is missing from the pasted text.**

So we cannot reproduce that missing block from your lab exactly.

The Apache configuration required for `mod_authn_dbd` uses:

- `DBDriver`
- `DBDParams`
- `AuthBasicProvider dbd`
- `AuthDBDUserPWQuery`

Apache's documentation confirms that `AuthDBDUserPWQuery` takes the username as the `%s` parameter and returns the password from the first column of the first matching row.

For your lab's MySQL database, the configuration should be:

```
DBDriver mysql
```

```
DBDParams "host=localhost dbname=apache user=root pass=cse"
```

```
<Directory "/var/www/example.com/html">
```

```
AuthType Basic
```

```
AuthName "Restricted Content"
```

```
AuthBasicProvider dbd
```

```
AuthDBDUserPWQuery "SELECT password FROM users WHERE username = %s"
```

```
    Require valid-user
</Directory>
```

Important

If your actual MySQL root password is **not** `cse`, replace:

```
pass=cse
```

with your actual MySQL password.

Better place for the database configuration

Put the database connection configuration in your Apache configuration, outside the `<Directory>` block.

For example:

```
DBDriver mysql
DBDParams "host=localhost dbname=apache user=root pass=cse"
```

```
<IfModule mod_ssl.c>
```

```
<VirtualHost *:443>
```

```
    ServerAdmin admin@example.com
    ServerName example.com
    ServerAlias www.example.com
```

```
    DocumentRoot /var/www/example.com/html
```

```
    ErrorLog ${APACHE_LOG_DIR}/example-error.log
    CustomLog ${APACHE_LOG_DIR}/example-access.log combined
```

```
    SSLEngine on
    SSLCertificateFile /etc/apache2/ssl/example.com.crt
    SSLCertificateKeyFile /etc/apache2/ssl/example.com.key
```

```
<Directory "/var/www/example.com/html">
    AuthType Basic
    AuthName "Restricted Content"
```

```
    AuthBasicProvider dbd
```

```
AuthDBDUserPWQuery "SELECT password FROM users WHERE username = %s"
```

```
Require valid-user  
</Directory>
```

```
</VirtualHost>
```

```
</IfModule>
```

Step 12 — Remove the **.htpasswd** configuration

This is important.

You previously had:

```
AuthUserFile /etc/apache2/.htpasswd
```

and:

```
Require valid-user
```

For Task 3, remove:

```
AuthUserFile /etc/apache2/.htpasswd
```

and replace the file-based provider with:

```
AuthBasicProvider dbd
```

and:

```
AuthDBDUserPWQuery "SELECT password FROM users WHERE username = %s"
```

So Task 2:

```
AuthType Basic  
AuthName "Restricted Content"  
AuthUserFile /etc/apache2/.htpasswd  
Require valid-user
```

becomes Task 3:

```
AuthType Basic  
AuthName "Restricted Content"  
AuthBasicProvider dbd
```

AuthDBDUserPWQuery "SELECT password FROM users WHERE username = %s"
Require valid-user

Step 13 — Test Apache

Run:

```
sudo apache2ctl configtest
```

Expected:

Syntax OK

Then:

```
sudo systemctl restart apache2
```

Check:

```
sudo systemctl status apache2
```

Expected:

active (running)

Step 14 — Test MySQL authentication

Open:

```
https://example.com
```

You should get:

Authentication Required

Username:

Password:

Enter:

Username: sammy

Password: sammys_password

Apache should query:

```
SELECT password FROM users
WHERE username = 'sammy';
```

and compare the submitted password with the stored {SHA} password.

The Apache `AuthDBUserPWQuery` mechanism is specifically designed to retrieve the password for the supplied username from SQL.

You can also test:

Username: alice

Password: alice_password



CHECKPOINT 3 — 8 Marks

Show your teacher:

<https://example.com>

Then:

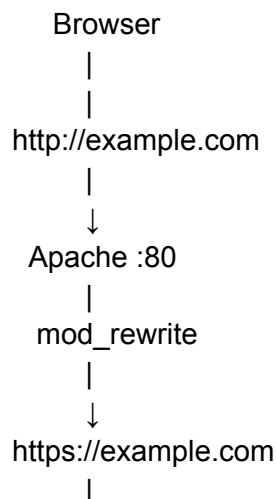
Username: sammy

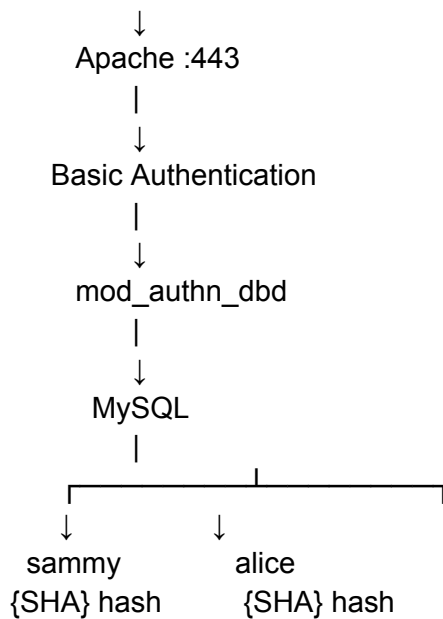
Password: *****

and demonstrate that the credentials stored in the **MySQL database** allow access.

Complete Lab Flow

After completing everything, your setup should look like this:





Checkpoint summary

Checkpoint	Task	What you demonstrate	Marks
1	Task 1	http://example.com → https://example.com	5
2	Task 2	<code>.htpasswd</code> authentication	7
3	Task 3	MySQL database authentication	8
Total			20

⚠ One thing I want you to do differently

Because you're working on **Ubuntu 24.04 + Apache/OpenSSL 3.x**, don't run all 20–30 commands at once. **Do Task 1 first and stop at `sudo apache2ctl configtest`.**

Run:

```
sudo a2enmod rewrite
sudo apache2ctl configtest
```

Then send me the **exact terminal output**. I'll guide you through the next step based on your actual Apache configuration, which is safer than guessing the virtual-host filename or structure.